Hierarchical Device–Edge–Cloud Task Offloading: Implementation Audit and Reproducible Simulation Baselines

Student 1 and Student 2
INSAT, Tunis, Tunisia
{student1,student2}@example.invalid

***Abstract*—This paper documents an auditable implementation of hierarchical task offloading across device, edge, and cloud tiers. The repository provides a discrete-event SimPy backend with separate device-placement, edge-placement, and cloud-queue decisions, shared resource queues, network serialization, and device energy accounting. Four Double-DQN adapters are integrated at the policy boundary, but their current formulation assigns terminal credit to each task decision and is therefore a contextual one-step formulation rather than a sequential multi-agent Gymnasium environment. The committed quantitative evidence is a three-seed, ten-task synthetic baseline campaign. Cloud forwarding reduced mean latency from 3.164 s for local execution to 0.513 s, while increasing communication traffic; all cloud and edge placements met the deadlines in this smoke workload, whereas local execution met 36.7% of deadlines. These results validate the execution and metric pipeline, not general policy superiority. Integrated-RL comparison artifacts, public-trace results, and final multi-edge learning experiments are not committed and are consequently reported as open validation work. The paper states the simulator assumptions, trace mapping, policy contracts, and limitations needed for reproducible follow-up experiments.**

***Index Terms*—mobile edge computing, task offloading, hierarchical reinforcement learning, discrete-event simulation, Double-DQN, reproducibility**

I. Introduction

Multi-access edge computing (MEC) places compute close to data producers while retaining access to larger cloud or HPC resources [1]. A task must therefore be mapped to a device CPU, an associated edge CPU, or a shared cloud queue. The choice couples execution time, queueing, wireless and backhaul transmission, device energy, and a relative deadline. A centralized controller can observe more state, but it also requires broader information sharing and can become a coordination bottleneck.

This project investigates a hierarchical alternative in which each tier makes only the decision exposed by its local contract. The device chooses local execution or its assigned edge; the edge chooses edge execution or cloud forwarding; and the cloud selects a queue discipline. The repository's contribution is primarily an auditable integration: a single SimPy backend executes all policies, with common task, timing, energy, and metric definitions. Double-DQN adapters and delayed adjacent-tier feedback are implemented at this boundary, but their scientific validation remains incomplete.

The article makes three bounded contributions. First, it specifies the three-tier model and policy interfaces. Second, it reports the committed three-seed synthetic baseline evidence without confusing it with trace measurements. Third, it identifies the exact gap between the implemented terminal one-step learning formulation and a sequential multi-agent RL environment. The remainder presents related work, the model, implementation, evaluation evidence, and limitations.

II. Related Work

MEC surveys emphasize that offloading decisions trade computation against communication and energy costs [2]. Dynamic formulations often combine placement, resource allocation, and time-varying queues; this project deliberately separates those concerns so that each tier has a small, inspectable action space. Unlike a full optimization formulation, the current backend is an independent-task simulator and does not claim optimality.

Simulation frameworks provide useful context for this design. EdgeCloudSim supports edge-network and computation experiments [3], while iFogSim targets fog/IoT resource-management modeling [4]. The repository instead uses a small custom SimPy model because device, edge, and cloud policy boundaries must remain separately injectable. This choice is documented as an engineering constraint, not as a claim that the model is more general than those frameworks.

Deep Q-learning provides a natural discrete-action baseline [5]. Double Q-learning reduces the overestimation tendency associated with naïve max targets [6]; the repository reuses this algorithmic family through independent device, edge, cloud, and centralized adapters. However, the current integration gives terminal credit to each task decision and does not expose a stepable Gymnasium multi-agent environment. It should consequently be compared to sequential hierarchical or multi-agent RL only after that environment and its temporal credit assignment are implemented.

Finally, Alibaba Cluster Trace 2017 is an infrastructure batch workload [7], not a mobile MEC measurement. The adapter derives compute demand from completed instance records, while payloads, deadlines, device assignment, battery, and network conditions remain synthetic. This distinction prevents a datacenter trace from being presented as direct wireless evidence.

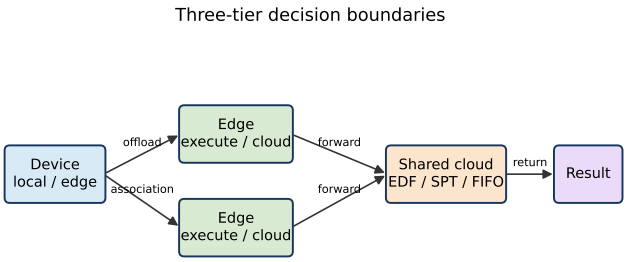


Fig. 1. Implemented decision boundaries and task route.

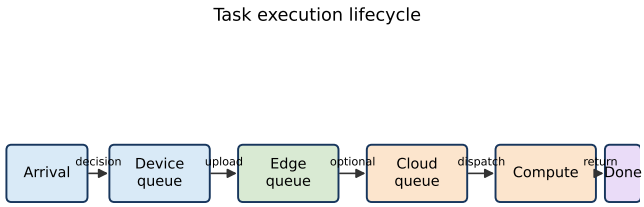


Fig. 2. Task lifecycle through queues, links, and result return.

III. Architecture and System Model

A. Three-tier topology

The canonical topology contains devices, associated edge servers, and one shared cloud scheduler. The default development configuration is one device–one edge–one cloud; the backend also accepts an explicit topology with three devices, two edges, and fixed device-to-edge associations. A task cannot choose an arbitrary edge. The explanatory architecture is shown in Fig. 1.

B. Task and resource model

Each task has arrival time, decimal-MB input and result sizes, required MI, a relative deadline, device identifier, and optional priority. Execution time is

$$t_{\text{cpu}} = \frac{\text{MI}}{\text{MIPS}}. \tag{1}$$

For a link with bandwidth B Mbps, propagation p, and queue wait q,

$$t_{\text{link}} = \frac{8s}{B} + p + q, \tag{2}$$

where s is decimal MB. End-to-end latency is completion minus arrival, and a deadline is met when completion is no later than arrival plus the relative deadline.

Every device and edge has a capacity-one SimPy resource. The cloud has one worker in the current implementation; its policy chooses EDF, shortest processing time, or FIFO when dispatching a queued job. Device energy uses CPU energy per MI and transmit/receive power times radio duration. Idle, reception infrastructure, retransmission, and cloud energy are not modeled.

C. Lifecycle and metrics

The lifecycle is arrival, device decision, optional device–edge upload, edge decision, optional edge–cloud upload and cloud dispatch, execution, and result return. Queues contribute waiting time. The result records route, timing, deadline status, device energy, communication delay and bytes. Fig. 2 illustrates the route without implying that all routes use cloud.

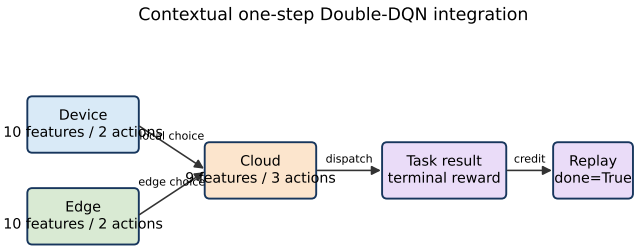


Fig. 3. Local observations and terminal per-task credit.

IV. Proposed Hierarchical Method

A. Policy contracts

The device observes normalized task demand, input size, deadline slack, battery fraction, local capacity, and device–edge link information, with an optional delayed edge-congestion signal. Its actions are local execution and edge offloading. The edge observes queue/utilization, task demand and deadline, edge–cloud link state, and an optional delayed cloud signal. Its actions are edge execution and cloud forwarding. The cloud observes queue length, deadline summaries, estimated execution time, and wait time; its actions are EDF, shortest processing time, and FIFO. The centralized comparator uses a 15-feature observation and selects device, edge, or cloud; cloud dispatch remains EDF.

B. Reward and learning formulation

For a task result r, the implemented team reward is

$$R = -\left(w_l \min(3, L/D) + w_e \min(3, E/E_0) w_d \mathbb{1}[\text{miss}] w_c \min(3, F/F_0)\right) \tag{3}$$

where L is latency, D the task deadline, E device energy, F feedback bytes, and (E_0, F_0) normalization references. The default weights are $(0.35, 0.25, 0.35, 0.05)$.

Each decision is stored with its own task identifier. Credit is assigned only after that task's terminal result is available, with a zero next observation and `done=True`. This is a contextual one-step learning formulation implemented with a Double-DQN replay learner, not a sequential multi-step MDP. Fig. 3 summarizes this distinction.

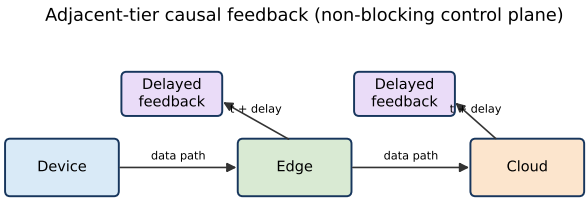


Fig. 4. Delayed adjacent-tier feedback; control traffic is non-blocking.

C. Coordination

Feedback is adjacent-tier and causal: a message sampled at time t becomes available after propagation plus serialization at a later decision. Modes are none, periodic, and frequent. Messages contribute recorded bytes and modeled delay, but they do not contend with data-plane resources or block execution. Thus the implementation measures control-plane accounting, not a complete communication-latency trade-off. Fig. 4 shows the implemented information path.

V. Implementation

The implementation targets Python 3.11 and uses SimPy 4.x, NumPy, Pandas, Matplotlib, PyYAML, Gymnasium interfaces at the RL boundary, PyTorch, and Stable-Baselines3-compatible project dependencies. The canonical simulator and domain layers do not import Gymnasium or Stable-Baselines3. Reproducibility is provided by explicit workload and policy seeds, fresh SimPy resources per run, CSV exports, and unit tests for models, metrics, queue order, observations, credit, feedback, and trace mapping.

A. Configuration and workload provenance

The synthetic smoke configuration uses three conceptual devices at 1000 MIPS, two demonstration edges at 500 and 750 MIPS, and a cloud configuration of 20,000 MIPS in the baseline protocol (the YAML multi-edge demonstration uses 500 MIPS). Device–edge bandwidth is 100 Mbps with 10 ms propagation, and edge–cloud bandwidth is 1000 Mbps with 40 ms propagation. Synthetic tasks use 0.05-s inter-arrivals, 250–1000 MI, 0.5–2.0 MB input, 0.05–0.2 MB result, and 1–4 s relative deadlines. These are assumptions, not trace measurements.

The committed baseline experiment uses the standard one-device/one-edge smoke configuration for comparability, three seeds, and ten tasks per seed. The Alibaba adapter is prepared for local 2017 `batch_task.csv` and `batch_instance.csv`; it derives MI from valid runtime and `real_cpu_avg`. No raw Alibaba files, processed trace result, or integrated-RL checkpoint is committed, so no trace or RL performance claim is made here.

TABLE I
Evidence status at the audited repository revision.

Capability	Status	Evidence boundary
Multi-device topology	implemented/tested	explicit topology code and tests; no final campaign
Multi-edge/shared cloud	implemented/tested	backend and topology configuration; no final campaign
Random, greedy, EDF	implemented/validated	committed three-seed synthetic CSV; controlled scenario tests
Device/edge/cloud RL	implemented/tested	adapters, observations, credit tests; no committed run
Centralized RL	implemented/tested	centralized adapter and tests; no committed run
Hierarchical coordination	implemented/tested	causal feedback unit tests; no final ablation
Alibaba workload	adapter/tested	mapping and loader tests; no committed trace results
Feedback overhead	modeled/tested	counters and theoretical delay; non-blocking

TABLE II
Three-seed synthetic baseline evidence (mean ± standard deviation).

Policy	Mean L (s)	P95 L (s)	Deadline	Energy (J)
Local	3.164 ± .467	5.491 ± .612	36.7%	.619 ± .072
Greedy/edge	.602 ± .118	.927 ± .137	100%	.565 ± .165
Edge execute	.602 ± .118	.927 ± .137	100%	.565 ± .165
Cloud/EDF	.513 ± .113	.745 ± .160	100%	.565 ± .165
Cloud/FIFO	.513 ± .113	.745 ± .160	100%	.565 ± .165
Random	1.177 ± .387	2.625 ± .761	90.0%	.424 ± .067

VI. Performance Evaluation

A. Evidence audit

Table I distinguishes source implementation, tests, and committed experimental evidence. This distinction is essential because a policy constructor or test does not establish a performance improvement.

B. Committed baseline results

The same seeded workload is passed to every policy. Table II reports the mean across three seeds and the sample standard deviation from `results/dev-smoke/aggregate_statistics.csv`.

Cloud forwarding has the lowest mean latency in this light synthetic workload, but it doubles the modeled payload traffic relative to edge execution (27.48 MB versus 13.74 MB per seed-average aggregate). Local execution consumes more device CPU energy and misses more deadlines. These observations are scenario-specific: greedy/edge equals edge execution because all default tasks satisfy the greedy offload inequality, and EDF equals FIFO because the smoke workload does not create a meaningful cloud ordering choice. The controlled contention scenario distinguishes their queue order but still misses its deliberately urgent task under both policies.

C. Unvalidated comparisons

The repository contains scripts for centralized RL, hierarchical RL, three feedback modes, and synthetic load/network scenarios. It does not contain the models, comparison CSVs, or run logs needed to reproduce those measurements at the

audited revision. Accordingly, no RL ranking, convergence, topology sensitivity, feedback benefit, or Alibaba improvement is reported. The required next experiment is to run training and held-out evaluation with disjoint seeds, then export the configured metrics and metadata alongside the commit and package versions.

VII. Conclusion and Perspectives

The repository currently supports a coherent three-tier SimPy execution model, separate device/edge/cloud policy contracts, explicit queues and links, device energy accounting, baseline metrics, and tested Double-DQN/feedback adapters. The committed baseline campaign is useful evidence that route and queue changes affect latency, deadlines, energy, and traffic; it is not evidence of universal superiority. Its main scientific limitation is that the integrated learning code assigns terminal credit per task and therefore does not yet constitute sequential hierarchical multi-agent RL.

Before a final claim, the project should run the integrated training and comparison scripts, archive checkpoints and metadata, evaluate a real local Alibaba 2017 sample with chronological splits, and perform matched multi-device/multi-edge experiments. Feedback should eventually contend for control-plane capacity if communication overhead is a research outcome. A multi-worker cloud, independent-task versus DAG workload comparison, calibrated energy model, confidence intervals, and a stepable Gymnasium interface are also required. Federated RL is future work rather than a current contribution.

References

[1] ETSI, "Multi-access edge computing (mec); framework and reference architecture," ETSI GS MEC 003 V2.2.1, 2020. [Online]. Available: https://www.etsi.org/deliver/etsi_gs/MEC/001_099/003/02.02.01_60/gs_MEC003v020201p.pdf

[2] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Communications Surveys and Tutorials*, vol. 19, no. 4, pp. 2322–2358, 2017.

[3] C. Sonmez, A. Ozgovde, and S. Ergen, "Edgecloudsim: An environment for performance evaluation of edge computing systems," in *2017 Second International Conference on Fog and Mobile Edge Computing*, 2017, pp. 39–44.

[4] H. Gupta, A. Vahid Dastjerdi, S. K. Ghosh, and R. Buyya, "ifogsim: A toolkit for modeling and simulation of resource management techniques in the internet of things, edge and fog computing environments," *Software: Practice and Experience*, vol. 47, no. 9, pp. 1275–1296, 2017.

[5] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[6] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double q-learning," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, no. 1, 2016.

[7] Alibaba Group, "Alibaba cluster trace 2017," ClusterData repository, 2018. [Online]. Available: https://github.com/alibaba/clusterdata/blob/master/cluster-trace-v2017/trace_201708.md